# Solution: Intervals Intersection

## Problem Statement

Given two lists of intervals, find the intersection of these two lists. Each list consists of disjoint intervals sorted on their start time.

**Example 1:**

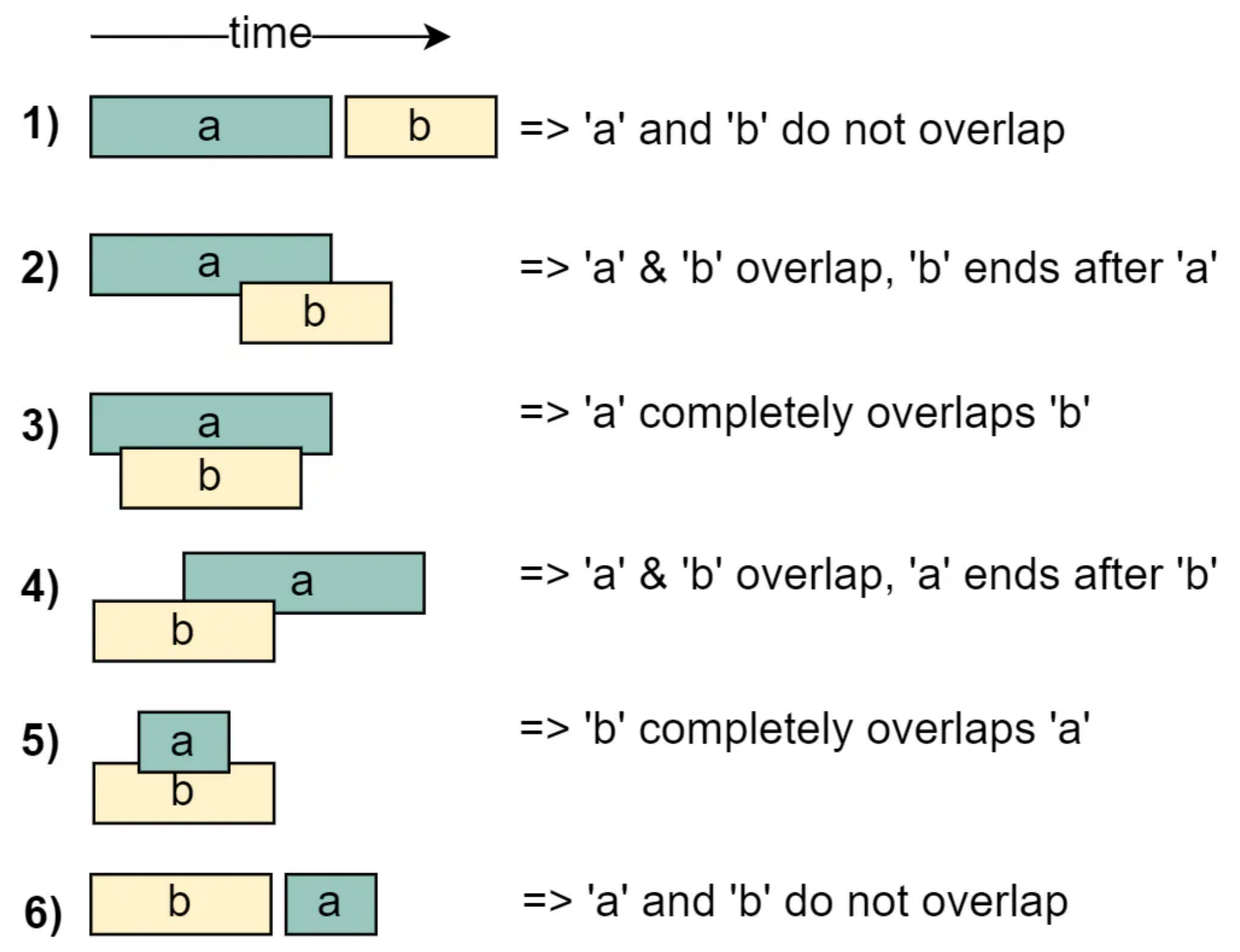
```
Input: arr1=[[1, 3], [5, 6], [7, 9]], arr2=[[2, 3], [5, 7]]
Output: [2, 3], [5, 6], [7, 7]
Explanation: The output list contains the common intervals between the
two lists.
```

**Example 2:**

```
Input: arr1=[[1, 3], [5, 7], [9, 12]], arr2=[[5, 10]]
Output: [5, 7], [9, 10]
Explanation: The output list contains the common intervals between the
two lists.
```

### Solution

This problem follows the `Merge Intervals` pattern. As we have discussed under `Insert Interval`, there are five overlapping possibilities between two intervals 'a' and 'b'. A close observation will tell us that whenever the two intervals overlap, one of the interval's start time lies within the other interval. This rule can help us identify if any two intervals overlap or not:


time
1) a b => 'a' and 'b' do not overlap
2) a b => 'a' & 'b' overlap, 'b' ends after 'a'
3) a b => 'a' completely overlaps 'b'
4) a b => 'a' & 'b' overlap, 'a' ends after 'b'
5) a b => 'b' completely overlaps 'a'
6) b a => 'a' and 'b' do not overlap

Now, if we have found that the two intervals overlap, how can we find the overlapped part?

Again from the above diagram, the overlapping interval will be equal to:

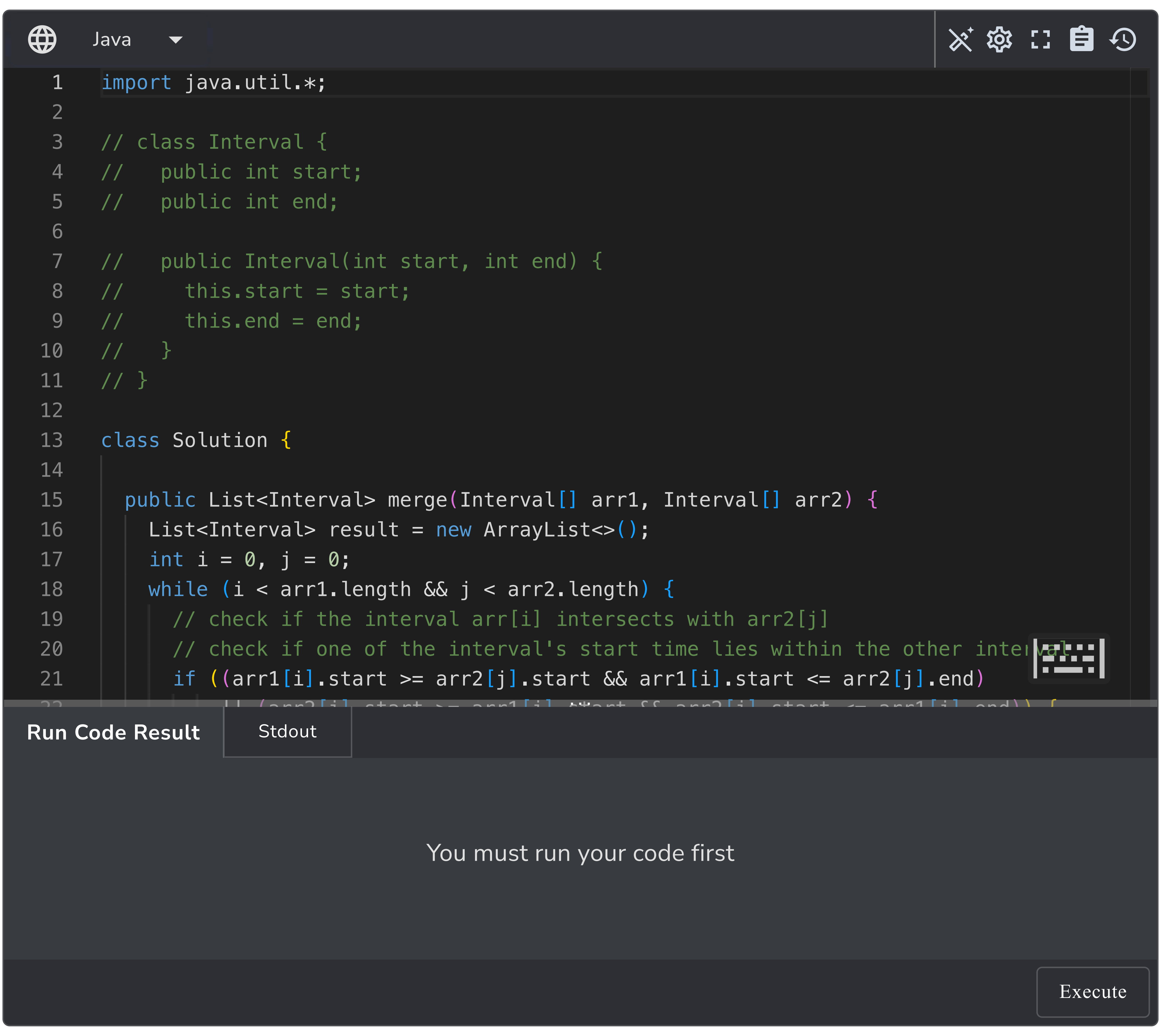
```
    start = max(a.start, b.start)
    end = min(a.end, b.end)
```

That is, the highest start time and the lowest end time will be the overlapping interval.

So our algorithm will be to iterate through both the lists together to see if any two intervals overlap. If two intervals overlap, we will insert the overlapped part into a result list and move on to the next interval which is finishing early.

## Code

Here is what our algorithm will look like:



```java
import java.util.*;

// class Interval {
//   public int start;
//   public int end;

//   public Interval(int start, int end) {
//     this.start = start;
//     this.end = end;
//   }
// }

class Solution {

  public List<Interval> merge(Interval[] arr1, Interval[] arr2) {
    List<Interval> result = new ArrayList<>();
    int i = 0, j = 0;
    while (i < arr1.length && j < arr2.length) {
      // check if the interval arr[i] intersects with arr2[j]
      // check if one of the interval's start time lies within the other inter
      if ((arr1[i].start >= arr2[j].start && arr1[i].start <= arr2[j].end)
```

You must run your code first

## Time Complexity

As we are iterating through both the lists once, the time complexity of the above algorithm is $O(N + M)$, where 'N' and 'M' are the total number of intervals in the input arrays respectively.

## Space Complexity

Ignoring the space needed for the result list, the algorithm runs in constant space O(1).

← Previous
Intervals Intersection (Medium)

Next →
Conflicting Appointments (Medium)

☐ Mark as Completed